Type Theory in a Lean Context

An Alternative Foundation to Mathematics

Naveen Maurya, IISc

October 12, 2026

Feynman Lecture Hall, ICTS

Table of contents

1. Foudational Systems

2. (Martin-Löf) Type Theory

3. Logic in Type Theory

4. Mathematics in Type Theory

5. Conclusion

Foudational Systems

What is a Foundational System?

Set Theory is not a "complete" foundation mathematics.

- Set theory has a problem about interpreting *types* of mathematical objects.
  For example, $\{\{\}, \{\{\}\}\}$ could mean either 2 or the ordered pair $(0, 0)$.
- Set theory doesn't center the type of organization mathematicians care about.
  For example, definitions of a *group* are closely related but that is not reflected in Set Theory.

What is a Foundational System?

A foundational system is an assumed framework to build on top of.

ZFC

What is a Foundational System?

Concretely, a Foundation System consists of

- A **Logical/Deductive system** specifying *rules of inference* that dictate which inferences are allowed from existing context.
- An **Axiomatic system**, a schematic collection of *axioms*, statements in the language of the deductive system, taken as true by assumption.

(Martin-Löf) Type Theory

Types, Terms and Universes

Types and *Terms* are primitive objects.

- Every term *a* is of some (unique) type, say α. We denote this $a : \alpha$.

Universes

Type Theory postulates a chain of cumulative universes in the style of Russell:

$$\mathcal{U}_0, \mathcal{U}_1, \mathcal{U}_2, \mathcal{U}_3, \ldots$$

satisfying:

- $\mathcal{U}_i : \mathcal{U}_{i+1}$ for each i
- If $A : \mathcal{U}_i$, then $A : \mathcal{U}_{i+1}$

Each type is a term of some universe. Each universe is also a term of universes above it.

Logic in Type Theory

Embedding a Logic through the Construction Material

Type Theory repurposes the available bricks for building math to also encode logic!

- Certain constructions and manipulations (type formers) of *bricks* are available in the system.

Certain types, called *mere propositions* may be used to capture logical propositions.
One may intepret terms of these types as *proofs* to the proposition.
Hence, proofs of statements are themselves first-class mathematical objects.

Type Formers

Type Former	Assumes	Notation	Logical Analogue
Function Type	$A, B : \mathcal{U}$	$A \to B$	$A \implies B$
Dependent Function Type	$A : \mathcal{U}, B : A \to \mathcal{U}$	$\prod_{(x:A)} B(x)$	$\forall x \in A, B(x)$
Dependent Pair Type	$A : \mathcal{U}, B : A \to \mathcal{U}$	$\sum_{(x:A)} B(x)$	$\exists x \in A, B(x)$
Product Type	$A, B : \mathcal{U}$	$A \times B$	$A \wedge B$
Coproduct Type	$A, B : \mathcal{U}$	$A + B$	$A \vee B$
Empty Type		$\mathbf{0}$	`False`
Unit Type		$\mathbf{1}$	`True`

Constructivity of Type Theory

Type Theory is a constructivist foundational system!

- A function $f : A \to B$ needs to have an alogorithm to effectively output an element $f(a) : B$ when given input $a : A$.
- $p \vee q$ requires either a proof of p or a proof of q.
- A proposition $\exists a : A, B(a)$ needs a an explicit $a : A$ such that $B(a)$ as a certificate.

Curry-Howard Correspondence b/w proofs and programs!
Type Theory requires proofs to be *computationally meaningful*.

Mathematics in Type Theory

Inductive Types

Natural numbers are the *model mathematical object* in Type Theory!

Remember Peano Axioms for $\mathbb{N}$,

- There is a distinguished element $0 \in \mathbb{N}$.
- There is an injective function $\mathtt{succ} : \mathbb{N} \to \mathbb{N}$,
- There is no $n \in \mathbb{N}$ such that $\mathtt{succ}(n) = 0$.
- The *induction principle*, i.e. to show a predicate P on $\mathbb{N}$ holds for all $n \in \mathbb{N}$, it suffices to prove $P(0)$ holds, and that for all $n \in \mathbb{N}, P(n) \implies P(\mathtt{succ}(n))$.

There's a somewhat obvious-looking property one can prove:

Theorem (Classification of Natural Numbers)
Any $n \in \mathbb{N}$ is either 0 or $\mathrm{succ}^m(0)$ for some $m \in \mathbb{N}$.

Proof.
Let $P(n)$ be the statement "$n = 0$ or there exists some $m \in \mathbb{N}$ such that $\mathrm{succ}^m(0) = n$."
By the induction principle, it suffices to show $P(0)$ and $\forall n \in \mathbb{N}, P(n) \implies P(\mathrm{succ}(n))$.

$P(0)$ is clearly true, and for some $n \in \mathbb{N}$, suppose $n = \mathrm{succ}^m(0)$, then note
$\mathrm{succ}(n) = \mathrm{succ}(\mathrm{succ}(m)(0)) = \mathrm{succ}^{\mathrm{succ}(m)}(0)$. □

Inductive Types

Here's a *universal property* about $\mathbb{N}$ wrt maps out of $\mathbb{N}$. Given $E : \mathbb{N} \to \mathcal{U}$,

$$\prod_{(n:\mathbb{N})} E(n) \simeq \left(E(0) \times \prod_{(n:\mathbb{N})} E(n) \to E(\mathtt{succ}(n)) \right)$$

Inductive Types

There is a general schema for constructing new types, requiring:

- **Formation rules** to introduce an inductive type into context.
- **Constructors/introduction rules** to introudce terms of the type.
- **Elimination rules** to consume terms of a type.
  (Usually by an *induction* principle which defines dependent functions on the type)
- **Computation rules** to specify action of elimination on the constructors.

These rules specify the behaviour of an inductive type by dictating the inference rules for the type in the foundational system.

Examples

Natural Numbers Type ($\mathbb{N}$) The natural numbers are an inductive type, given by the constructors

- $\mathtt{zero} : \mathbb{N}$
- $\mathtt{succ} : \mathbb{N} \to \mathbb{N}$

and the induction principle that for $E : \mathbb{N} \to \mathcal{U}$, to prove $\prod_{(n:\mathbb{N})} E(n)$ is equivalent to proving $E(0)$ and $\prod_{(n:\mathbb{N})} E(n) \to E(\mathtt{succ}(n))$. Formally,

$$\mathbf{ind}_{\mathbb{N}} : \prod_{(E:\mathbb{N}\to\mathcal{U})} (E(0) \times \prod_{(n:\mathbb{N})} (E(n) \to E(\mathtt{succ}(n)))) \to \prod_{(n:\mathbb{N})} E(n)$$

Examples

The Empty Type (0) The empty type $\mathbf{0}$ is given by no constructors. One has the universal function for any $E : \mathbf{0} \to \mathcal{U}$, one has the universal function $\prod_{(z:\mathbf{0})} E(z)$. Formally,

$$\mathbf{ind}_0 : \prod_{(E:\mathbf{0}\to\mathcal{U})} \prod_{(z:\mathbf{0})} E(z)$$

Examples

The Unit Type (1) The unit type $\mathbf{1}$ is given by one constructor,

- $* : \mathbf{1}$.

and the induction principle that for $E : \mathbf{1} \to \mathcal{U}$, to prove $\prod_{(z:\mathbf{1})} E(z)$, it suffices to prove $E(*)$. Formally,

$$\mathbf{ind}_1 : \prod_{(E:\mathbf{1}\to\mathcal{U})} E(*) \to \prod_{(z:\mathbf{1})} E(z)$$

Examples

The Coproduct Type ($A + B$) The coproduct type of A and B is an inductive type, given by the constructors

- $\mathtt{inl} : A \to A + B$
- $\mathtt{inr} : B \to A + B$

and the induction principle that for $E : A + B \to \mathcal{U}$, to prove $\prod_{(z:A+B)} E(z)$ is equivalent to proving $E(\mathtt{inl}(a))$ for all $a : A$ and $E(\mathtt{inr}(b))$ for all $b : B$. Formally,

$$\mathbf{ind}_{A+B} : \prod_{(E:A+B\to\mathcal{U})} \left(\prod_{(a:A)} E(\mathtt{inl}(a)) \times \prod_{(a:A)} E(\mathtt{inl}(a)) \to \prod_{(z:A+B)} E(z) \right)$$

A Custom Inductive Type

Binary Trees Given a type $\mathtt{A} : \mathcal{U}$, the type of binary trees over $\mathtt{A}$ is $\mathtt{BinTree}(\mathtt{A})$ given by the constructors

- $\mathtt{leaf} : \mathtt{A} \to \mathtt{BinTree}(\mathtt{A})$
- $\mathtt{node} : \mathtt{BinTree}(\mathtt{A}) \to \mathtt{BinTree}(\mathtt{A}) \to \mathtt{BinTree}(\mathtt{A})$

with the induction principle, that for $E : A \to \mathcal{U}$, to prove $\prod_{(z:\mathtt{BinTree}(\mathtt{A}))} E(z)$, it suffices to prove $\prod_{(a:A)} E(\mathtt{leaf}(\mathtt{a}))$ and $\prod_{(t_1,t_2:\mathtt{BinTree}(\mathtt{A}))} E(\mathtt{node}(t_1)(t_2))$.

Structural Constructions

Magma A *magma* is a type M with a binary operation $\cdot : M \times M \to M$. The type *Magma* is given by

$$\mathit{Magma} :\equiv \sum_{(M:\mathcal{U})} M \times M \to M$$

Semigroup A *semigroup* is a magma with the operation being associative

$$\mathit{Semigroup} :\equiv \sum_{(M:\mathcal{U})} \sum_{(\cdot:M\times M\to M)} \prod_{(a,b,c:M)} (a \cdot b) \cdot c = a \cdot (b \cdot c)$$

Identity Types

What does it mean for two mathematical objects to be equal?
In (intensive) type theory, there are two types of equalities:

- **Definitional Equality** Equality within the logical framework.
- **Propositional Equality** A genuine equality of defined mathematical objects.

Identity Types

What does it mean for two mathematical objects to be equal?
In (intensive) type theory, there are two types of equalities:

- **Definitional Equality** Equality within the logical framework.
- **Propositional Equality** A genuine equality of defined mathematical objects.

Remark. An equality $p : A =_{\mathcal{U}} B$ in a universe gives a transport $p_* : E(A) \to E(B)$ for any $E : A \to \mathcal{U}$.

Conclusion

Conclusion

References

Thank You!